Twin Prime Infinitude by Finite Contradiction

alegator (coideated with GPT 5.4, 5.5)

May 6, 2026

Abstract

We formalize an unconditional twin prime infinitude theorem dependent only on standard axioms and an externally proven finite certificate. This is by manner that supposing the finiteness of twins forces a cofinal tail of "exceptional" primes, which recursively bubble down composite-prime and prime-composite pair counts into a preceding finite block, necessarily eventually exceeding its capacity for such pairs, forcing a contradiction by set cardinality. This is proven for a finite base case by finding the first adjacent exception tail after the largest known exception $B = 127$ that would cause the contradiction, and by induction, all further exceptions also contradict, and the initial finite twins hypothesis is false, so infinite twins is proven.

1 Introduction

This proof is written and formalized in Lean4 by a "citizen mathematician" with the coideation of GPT 5.4 and 5.5 through web client and Codex interfaces. The twin prime conjecture is the 2-gap case of de Polignac's 1849 conjecture on even prime gaps [1]. Brun proved in 1919 that the reciprocal series over twin primes converges, introducing sieve methods into the problem [2]. Chen proved that infinitely many primes p have $p + 2$ either prime or the product of at most two primes [3]. The modern bounded-gap breakthrough began with Zhang's finite bound for gaps between consecutive primes [4], was sharpened by the Maynard–Tao method [5], and was pushed by Polymath8 to the unconditional bound $H_1 \leq 246$ [6].

This paper does not try to improve sieve weights or follow the bounded-gap $b \leq 246$ route. It uses midpoint rows and a finite recursive certificate: if twins were finite, a cofinal tail of exceptional primes would force composite-prime and prime-composite middle-pair events to descend into a fixed finite block. The generated recursive closure predicts more distinct events than the block contains. Lean checks the set-theoretic contradiction and the reduction from no cofinal exceptional tail to arbitrarily large twin primes; the external C++ certificate is the finite descent and counting boundary.

Contents

2 Overview

2.1 Midpoint rows and exceptional primes

The proof endpoint studied here is organized around a very small object. For a prime r, inspect the midpoint row

$$M_r = \{rh : 1 \le h < r\}.$$

The midpoint rh gives the candidate twin pair

$$rh - 1, \qquad rh + 1.$$

If some h in the row gives two primes, then we have a twin-prime pair. A prime r is called an exception, or a midpoint-exceptional prime, if no such index exists.

The project uses this exception object to turn twin infinitude into a first-exception-after-a-bound problem. If twin primes were eventually absent, then for every sufficiently large prime r, all row midpoints rh would lie beyond the last twin midpoint. Then every sufficiently large prime would be exceptional. A cofinal tail of exceptional primes has a first exceptional prime after any chosen finite verified bound. To prove infinitely many twins, it is enough to rule out such a cofinal exceptional tail.

Empirically, the last small exception in the larger search was 127, and it was tested that the next gap is at least

$$V_0 = 191264$$

2.2 Split primes and recursive MP/PM closure

In this certificate, a split prime is a prime for which 5 is a quadratic residue modulo that prime:

$$\exists s, \qquad s^2 \equiv 5 \pmod{p}.$$

The row congruences expose this residue directly. For left rows,

$$4\big(u(u+3)+1\big) = (2u+3)^2 - 5,$$

so $p \mid u(u+3)+1$ forces

$$(2u+3)^2 \equiv 5 \pmod{p}.$$

For right rows,

$$4\big((u+1)(u+4)+1\big) = (2u+5)^2 - 5,$$

so $p \mid (u+1)(u+4)+1$ forces

$$(2u+5)^2 \equiv 5 \pmod{p}.$$

The labels MP and PM denote the two side-labeled finite event classes in the generated middle-pair block: one side records composite-prime middle-pair events and the other records prime-composite middle-pair events.

The recursive MP/PM event closure is the finite descent operation used by the external certificate. For a split prime p, the program constructs a direct finite target-event set $T(p)$, a finite descended prime set $D(p)$ consisting only of smaller split primes, and the recursively reachable event set

$$R(p) = T(p) \cup \bigcup_{q \in D(p)} R(q).$$

The inequality $q < p$ makes this a well-founded finite recursion. The core certificate implication is that a cofinal exceptional tail makes every event in the computed closure actual in the finite MP/PM block.

The row formulas used by this closure are the two width-two quadratic identities. A left row is an integer u for which

$$p \mid u(u+3) + 1.$$

With

$$h = \frac{u(u+3)+1}{p},$$

one has

$$ph - 1 = u(u+3), \qquad ph + 1 = (u+1)(u+2),$$

using

$$u(u+3) + 2 = (u+1)(u+2).$$

A right row is an integer u for which

$$p \mid (u+1)(u+4) + 1.$$

With

$$h = \frac{(u+1)(u+4)+1}{p},$$

one has

$$ph - 1 = (u+1)(u+4), \qquad ph + 1 = (u+2)(u+3),$$

using

$$(u+1)(u+4) + 2 = (u+2)(u+3).$$

The discriminant of both row congruences is 5, which is why the certificate uses split primes.

The five adjacent slots

$$u,\ u+1,\ u+2,\ u+3,\ u+4$$

are then factored. Any split prime factor $q < p$ in those slots is a descended prime route. Recursion continues from q. The generated computation observes that descent into the early finite MP/PM block is overwhelmingly the common case; the recursive closure formalizes this by exhaustively following all smaller descended primes until the decreasing graph bottoms out. The following table is one concrete route printed by

`tools/print_routing_example.cpp`.

stage	prime	row	quadratic data	next event
1	191281	$R,\ u = 183108,\ h = 175289$	$191281 \cdot 175289 = (183109)(183112) + 1$	$u + 2 = 183110 = 10 \cdot 18311$
2	18311	$R,\ u = 1486,\ h = 121$	$18311 \cdot 121 = (1487)(1490) + 1$	$u + 3 = 1489$
3	1489	$L,\ u = 807,\ h = 439$	$1489 \cdot 439 = 807 \cdot 810 + 1$	$u + 4 = 811$
4	811	$R,\ u = 780,\ h = 755$	$811 \cdot 755 = 781 \cdot 784 + 1$	$u + 1 = 781 = 11 \cdot 71$
5	71	$R,\ u_0 = 6,\ h = 1$	$71 = (7)(10) + 1$	$u_t = 1426 \equiv 6 \pmod{71},\ u_t \in \mathrm{MP} \cap \mathrm{PM}$

This is a typical closure move: each row produces a smaller split prime through one of its five slots, and the final descended row reaches the early finite MP/PM block. Distinct routes can collide at the same side-labeled target event, and some routes can escape the target block or fail to produce a descended split prime before bottoming out. The certificate is built to handle both effects: it counts only unique predicted side-labeled MP/PM pairs in the target block, so duplicate routes do not inflate E_p, and escaping routes are simply absent from the predicted target-event set.

The routing implication used by the proof is exactly this: under a cofinal tail of exceptional primes, every predicted MP/PM pair reached by the recursive descent is forced to be an actual MP/PM pair in the target block. The finite contradiction comes from producing more unique predicted target pairs than the target block actually contains.

2.3 Finite contradiction overview

The external finite contradiction certificate starts at the first split prime

$$R_0 = 191281.$$

Let

$$E_p = 181052$$

be the number of distinct predicted side-labeled MP/PM events produced by the recursive finite computation, and let

$$E_a = 95568$$

be the size of the actual generated MP/PM event set. Since $E_p > E_a$, a cofinal exceptional tail realizing those predicted events is impossible. The generated C++ certificate packages precisely the route-realization bridge as the Lean declaration

`GeneratedCertificate.external_predictedEvents_realized_of_cofinalTail`.

Lean wraps the generated finite sets in

`RecursiveMPPMEventCertificate`

and derives

`GeneratedCertificate.external_no_cofinalExceptionTail`

from the bridge and the checked strict inequality $E_a < E_p$.

The proof in Lean is intentionally transparent. Everything after this one external route-realization bridge is ordinary Lean proof. The final theorem is

`TwinPrimeExternal.arbitrarily_large_twins`.

Its statement expands to:

$$\forall N,\ \exists p,\ N \le p \wedge \mathrm{Prime}(p) \wedge \mathrm{Prime}(p+2).$$

2.4 What is external and what is Lean-checked

The final theorem depends on the standard Lean axioms

`propext`, `Classical.choice`, `Quot.sound`,

and on exactly one project-specific external route-realization certificate:

`GeneratedCertificate.external_predictedEvents_realized_of_cofinalTail`.

The theorem

`GeneratedCertificate.external_no_cofinalExceptionTail`

is Lean-derived from

`no_cofinalExceptionTail_of_recursiveMPPMEventCertificate`.

The auxiliary gap certificate

`GeneratedGapCertificate.external_exceptionFree_to_certificateThreshold`

proves that no exceptional prime after 127 occurs at or before 191264, but the final theorem does not depend on it. It is included to document the empirical starting point.

2.5 Formal object directory for Section 1

`TwinPrimeExternal/Core.lean`

- `lastObservedException`: the last observed exceptional prime, 127.
- `observedVerifiedTo`: the larger motivating verified bound, 10^9.
- `certificateVerifiedTo`: the reduced certificate threshold, 191264.
- `certificateFirstSplitPrime`: the first split prime used by the recursive certificate, 191281.
- `generatedMPPMCard`: the generated actual MP/PM event count, 95568.
- `predictedEventCount`: the generated predicted event count, 181052.
- `predictedEventCount_exceeds_generatedMPPMCard`: checks $95568 < 181052$.
- `ObservedExceptionGap`: the record for a last-exception bound and a later verified bound.
- `ObservedExceptionGap.size`: the numerical size of such a gap.
- `observedExceptionGap`: the 127 to 10^9 motivating gap record.
- `certificateExceptionGap`: the 127 to 191264 certificate gap record.
- `certificateVerifiedTo_le_observed`: compares the reduced certificate gap with the larger observed gap.

`TwinPrimeExternal/RecursiveMPPMCertificate.lean`

- `left_row_residue5_identity`: the left-row residue-5 identity.
- `right_row_residue5_identity`: the right-row residue-5 identity.

- `quad_route_left_identity`: the left width-two routing identity.
- `quad_route_right_identity`: the right width-two routing identity.
- `encodeSideEvent`: encodes a target coordinate and side label as one natural number.
- `encodeSideEvent_zero_ne_one`: separates the two side labels at a fixed coordinate.
- `encodeSideEvent_injective_of_side_lt_two`: proves injectivity for side labels $0, 1$.
- `DescendedPrimeEdge`: the decreasing-edge predicate for descended primes.
- `DescendedPrimeEdge.decreases`: extracts the strict decrease from a descended edge.
- `DescendedPrimeEdge.irrefl`: rules out a self-edge.
- `DescendedPrimeEdge.no_two_cycle`: rules out a two-cycle.

`TwinPrimeExternal/GeneratedCertificate.lean`

- `E_a`: the generated actual event count.
- `E_p`: the generated predicted event count.
- `E_p_exceeds_E_a`: checks the generated overflow inequality.
- `external_predictedEvents_realized_of_cofinalTail`: the final-theorem-relevant external route-realization declaration.

`TwinPrimeExternal/Final.lean`

- `arbitrarily_large_twins`: the final exported Lean endpoint.

3 Midpoint-exceptional primes

Definition 3.1 (Row index)**.** *For $r, h \in \mathbb{N}$, define*

$$\text{MidpointRowIndex}(r, h) \quad :\Longleftrightarrow \quad 1 \le h \ \wedge \ h < r.$$

This is Lean definition

MidpointRowIndex.

Definition 3.2 (Midpoint and twin witness)**.** *For $r, h \in \mathbb{N}$, define the midpoint*

$$\text{MidpointRowMidpoint}(r, h) = rh.$$

Define

$$\text{MidpointTwinWitnessAt}(r, h) \quad :\Longleftrightarrow \quad \text{Prime}(rh - 1) \ \wedge \ \text{Prime}(rh + 1).$$

These are Lean definitions

MidpointRowMidpoint, *MidpointTwinWitnessAt.*

Definition 3.3 (Midpoint-exceptional prime)**.** *A prime r is a midpoint-exceptional prime, or simply an exception, if*

$$\text{Prime}(r) \quad \textit{and} \quad \forall h, \ (1 \le h < r) \Rightarrow \neg\, \text{MidpointTwinWitnessAt}(r, h).$$

In Lean this is

MidpointExceptionalPrime.

This direct definition says that the midpoint row associated to r contains no twin-prime pair. Earlier searches also used divisor formulations of exception. The finite gap checker in Appendix A bridges the divisor view to this direct midpoint-row view in a bounded window.

3.1 Formal objects used in Section 2

`TwinPrimeExternal/Core.lean`

- `MidpointRowIndex`: the row-index predicate $1 \le h < r$.
- `MidpointRowMidpoint`: the midpoint rh.
- `MidpointTwinWitnessAt`: the predicate that $rh - 1$ and $rh + 1$ are both prime.
- `MidpointExceptionalPrime`: the definition of a midpoint-exceptional prime.

4 From finite twins to an exceptional tail

Definition 4.1 (Bounded twin midpoints)**.** *The predicate* BoundedTwinMids *says that there exists a bound M such that no midpoint $m > M$ has both $m-1$ and $m+1$ prime:*

$$\exists M,\ \forall m > M,\ \neg(\mathrm{Prime}(m-1) \wedge \mathrm{Prime}(m+1)).$$

This is Lean definition

BoundedTwinMids.

Definition 4.2 (Arbitrarily large twins)**.** *The target theorem is*

$$\mathrm{ArbitrarilyLargeTwins} \quad :\Longleftrightarrow \quad \forall N,\ \exists p,\ N \le p \wedge \mathrm{Prime}(p) \wedge \mathrm{Prime}(p+2).$$

This is Lean definition

ArbitrarilyLargeTwins.

Lemma 4.3 (Bounded midpoints force a cofinal exceptional tail)**.** *If* BoundedTwinMids *holds, then there exists B such that every prime $r > B$ is a* MidpointExceptionalPrime*.*

Proof. Choose M witnessing BoundedTwinMids. Set $B = M$. Let r be prime and suppose $M < r$. To prove that r is exceptional, let h satisfy $1 \le h < r$. The corresponding midpoint is

$$m = rh.$$

Since $h \ge 1$,

$$r = r \cdot 1 \le rh = m.$$

So $M < r \le m$, so $M < m$. By the defining property of M, the pair $(m-1, m+1)$ is not a twin-prime pair. So no row index h gives a twin pair, so r is a MidpointExceptionalPrime. □

This is Lean theorem

`boundedTwinMids_forces_cofinalTail_midpointExceptionalPrime`.

4.1 Formal objects used in Section 3

`TwinPrimeExternal/Core.lean`

- `BoundedTwinMids`: the bounded-twin-midpoint hypothesis.
- `ArbitrarilyLargeTwins`: the target infinitude predicate.
- `CofinalExceptionTail`: packages that every prime past a bound is exceptional.
- `boundedTwinMids_forces_cofinalTail_midpointExceptionalPrime`: proves that bounded twin midpoints force a cofinal exceptional tail.
- `arbitrarily_large_twins_of_not_boundedTwinMids`: proves the elementary endpoint from the negation of bounded twin midpoints.

5 First exceptions after a bound

Definition 5.1 (Observed exception gap)**.** *An* ObservedExceptionGap *is a record containing a last exception bound and a later verified bound:*

$$(\text{lastException}, \text{verifiedTo}), \qquad \text{lastException} < \text{verifiedTo}\,.$$

The certificate gap in this project is

$$\text{lastException} = 127, \qquad \text{verifiedTo} = 191264.$$

In Lean:

certificateExceptionGap.

Definition 5.2 (First exception after a gap)**.** *For an exception predicate* $E : \mathbb{N} \to \text{Prop}$ *and a gap* γ*, a value*

$$\text{FirstExceptionAfterGap}(E, \gamma)$$

consists of a prime r *such that:*

(i) $r > \gamma.\,\text{verifiedTo}$*;*

(ii) $E(r)$*;*

(iii) no prime s *with*

$$\gamma.\,\text{verifiedTo} < s < r$$

satisfies $E(s)$*.*

This is Lean structure

FirstExceptionAfterGap.

Lemma 5.3 (A cofinal exceptional tail gives a first exception after any gap)**.** *Let* $E : \mathbb{N} \to \text{Prop}$*. If there exists* B *such that every prime* $r > B$ *satisfies* $E(r)$*, then for every observed gap* γ*, there exists a* $\text{FirstExceptionAfterGap}(E, \gamma)$*.*

Proof. Choose a prime $r_0 > \max(B, \gamma.\,\text{verifiedTo})$, using infinitude of primes. Then $E(r_0)$, so the set

$$S = \{p : \ p \text{ prime}, \ \gamma.\,\text{verifiedTo} < p, \ E(p)\}$$

is nonempty. By well-ordering of $\mathbb{N}$, let R be its least element. Then R is prime, $R > \gamma.\,\text{verifiedTo}$, and $E(R)$. By minimality, no smaller prime s above $\gamma.\,\text{verifiedTo}$ satisfies $E(s)$. So R is the first exceptional prime after γ's verified bound. □

This is Lean theorem

`cofinalTail_gives_firstExceptionAfterGap`.

Lemma 5.4 (Gap monotonicity)**.** *If* $\gamma_1.\,\text{verifiedTo} \le \gamma_2.\,\text{verifiedTo}$*, then any first exception after* γ_2 *induces a first exception after* γ_1*.*

Proof. Let R be the first exception after γ_2. Then R is an exception prime above $\gamma_2.\,\mathrm{verifiedTo}$, so also above $\gamma_1.\,\mathrm{verifiedTo}$. Take the least exception prime above $\gamma_1.\,\mathrm{verifiedTo}$. This least prime is the required first exception after γ_1. □

This is Lean theorem

`firstException_mono_backward`.

The contrapositive, used for forwarding no-first-exception results to later gaps, is

`no_firstException_mono_forward`.

5.1 Formal objects used in Section 4

`TwinPrimeExternal/Core.lean`

- `ObservedExceptionGap`: the gap record.
- `ObservedExceptionGap.size`: the numerical size of a gap.
- `lastObservedException`: the last observed exceptional prime.
- `certificateVerifiedTo`: the reduced certificate threshold.
- `observedVerifiedTo`: the larger motivating verified threshold.
- `certificateExceptionGap`: the concrete reduced certificate gap.
- `observedExceptionGap`: the concrete motivating observed gap.
- `certificateVerifiedTo_le_observed`: compares the certificate gap with the larger observed gap.
- `FirstExceptionAfterGap`: the first-exception-after-a-gap record.
- `FirstExceptionAfterLastObserved`: the version used by the auxiliary gap checker.
- `ExceptionFreeUpTo`: the finite no-exception predicate.
- `firstExceptionAfterLast_occurs_after_of_exceptionFreeUpTo`: pushes a first exception past a checked window.
- `firstExceptionAfterLast_occurs_after_certificateThreshold_of_exceptionFreeUpTo`: the certificate-threshold specialization.
- `cofinalTail_gives_firstExceptionAfterGap`: extracts a first exception after a chosen gap from a cofinal exceptional tail.
- `firstException_mono_backward`: first-exception monotonicity toward earlier gaps.
- `no_firstException_mono_forward`: the no-first-exception contrapositive toward later gaps.

6 The external finite certificate

The main external finite certificate is expressed in Lean as the route-realization bridge

- `GeneratedCertificate.external_predictedEvents_realized_of_cofinalTail`

whose mathematical shape is

$$\big(\exists B, \mathrm{CofinalExceptionTail}(\mathrm{MidpointExceptionalPrime}, B)\big) \Rightarrow \mathrm{predictedEvents} \subseteq \mathrm{actualEvents}\,.$$

Lean combines this bridge with the finite event counts to prove:

- `GeneratedCertificate.external_no_cofinalExceptionTail`

whose mathematical shape is

$$\text{Not } \exists B,\ \mathrm{CofinalExceptionTail}(\mathrm{MidpointExceptionalPrime}, B).$$

The generated file records the numerical overflow:

$$E_a = 95568,$$

$$E_p = 181052,$$

and Lean checks

$$E_a < E_p.$$

The Lean theorem name for the arithmetic inequality is

`GeneratedCertificate.E_p_exceeds_E_a`.

Certificate 6.1 (Generated finite contradiction obstruction)**.** *There is no cofinal tail of midpoint-exceptional primes:*

$$\text{Not } \exists B,\ \mathrm{CofinalExceptionTail}(\mathrm{MidpointExceptionalPrime}, B).$$

External certificate proof. The C++ program `search_recursive_prefix_threshold.cpp` computes the finite recursive MP/PM event closure from the generated MP/PM block. It finds a finite prefix beginning at 191265 whose first split prime is 191281 and whose recursive closure contains 181052 distinct side-labeled predicted MP/PM events. The generated finite MP/PM event set contains only 95568 events. Under a cofinal exceptional tail, the finite closure makes every predicted event actual in the side-labeled MP/PM event set. This would inject a set of cardinality 181052 into a set of cardinality 95568, impossible.

The correctness argument for the finite computation is included below in this section. In Lean, the route-realization part is the external declaration

`GeneratedCertificate.external_predictedEvents_realized_of_cofinalTail`.

The cardinal contradiction from this bridge is checked by

`no_cofinalExceptionTail_of_recursiveMPPMEventCertificate`.

□

6.1 Correctness of the recursive MP/PM event certificate

This subsection proves the correctness of

`tools/search_recursive_prefix_threshold.cpp`.

The purpose of the algorithm is finite and exact: compute the recursive quadratic-routing closure of selected split primes into a generated side-labeled MP/PM event universe, and stop when the number of distinct predicted events exceeds the actual MP/PM cap.

6.1.1 Input certificate

The input JSON file

`certificates/generated_mppm_pressure_certificate.json`

contains three finite arrays:

$$\text{Base}, \qquad \text{MP}, \qquad \text{PM}.$$

The algorithm builds a side-labeled predicted event universe

$$U = \{(u, 0) : u \in \text{Base}\} \cup \{(u, 1) : u \in \text{Base}\},$$

encoded by

$$\text{encode}(u, s) = 2u + s.$$

Here $s = 0$ is the MP-side label and $s = 1$ is the PM-side label. The encoding is just parity bookkeeping: the side label is recovered from the parity of $2u + s$, and the coordinate u is recovered by integer division by 2. In Lean this is the function

`encodeSideEvent`.

The theorems

`encodeSideEvent_zero_ne_one`

and

`encodeSideEvent_injective_of_side_lt_two`

record that the two side labels for a fixed u are distinct and that the encoding is injective when $s < 2$. It builds the actual side-labeled MP/PM set

$$A = \{(u, 0) : u \in \text{MP}\} \cup \{(u, 1) : u \in \text{PM}\}.$$

The generated data gives

$$|A| = 95568.$$

6.1.2 Split primes and rows

For a prime p, the predicate `leg5(p)` tests whether 5 is a quadratic residue modulo p. If not, the prime has no rows.

If 5 is a quadratic residue modulo p, the function `sqrt5_mod_prime(p)` returns the two square roots of 5 modulo p. This is computed either by the $p \equiv 3 \pmod 4$ formula or by Tonelli-Shanks [7]. Both branches are standard algorithms satisfying:

$$s \in \texttt{sqrt5\_mod\_prime(p)} \quad \Longleftrightarrow \quad s^2 \equiv 5 \pmod p.$$

For each such s, the algorithm builds two row types. The right row solves

$$(u+1)(u+4)+1 \equiv 0 \pmod p,$$

and the left row solves

$$u(u+3)+1 \equiv 0 \pmod p.$$

The formulas in `rows_for(p)` are exactly the quadratic formula for these two congruences. Whenever the computed midpoint is divisible by p, the algorithm sets

$$h = \frac{(u+1)(u+4)+1}{p} \quad \text{or} \quad h = \frac{u(u+3)+1}{p},$$

and retains the row only when

$$1 \le h < p.$$

So every retained row is a valid midpoint-row index for p, and the row records the five adjacent slots

$$u, u+1, u+2, u+3, u+4.$$

Lean checks the residue and row identities used here as

`left_row_residue5_identity`, `right_row_residue5_identity`,
`quad_route_left_identity`, `quad_route_right_identity`.

6.1.3 Target events

The function `target_events_at(p)` maps rows at p into the finite event universe U. If $p < X$, the row coordinate u is lifted through the congruence class

$$u,\ u+p,\ u+2p, \ldots < X.$$

For each lifted u, the side-labeled events $(u, 0)$ and $(u, 1)$ are looked up in U. If present, their integer identifiers are inserted. The result is sorted and deduplicated. So `target_events_at(p)` returns exactly the finite set of generated Base-side events hit directly by rows at p.

6.1.4 Descended prime edges and recursive closure

For a retained row at p, the algorithm factors each of the five slot values $u, u+1, u+2, u+3, u+4$. A prime factor q is accepted as a descended prime if

$$31 \le q < p \quad \text{and} \quad 5 \text{ is a quadratic residue modulo } q.$$

The strict inequality $q < p$ is essential: descended prime edges always go to smaller primes, so the graph is acyclic. The reduced Lean model records this local fact as

- `DescendedPrimeEdge.decreases`
- `DescendedPrimeEdge.irrefl`
- `DescendedPrimeEdge.no_two_cycle`

The full finite graph construction remains part of the external C++ route audit.

Define the finite recursive closure $R(p)$ by well-founded recursion on prime size:

$$R(p) = T(p) \cup \bigcup_{q \in D(p)} R(q),$$

where $T(p)$ is the direct target-event set and $D(p)$ is the finite descended prime set. The function `reachable_events(p)` implements exactly this recursion, using memoization. Memoization does not alter the mathematical value; it only caches already-computed $R(q)$. Since every descended prime satisfies $q < p$, the recursion terminates.

6.1.5 Primality and factorization subroutines

The C++ certificate uses deterministic 64-bit Miller-Rabin for primality testing and Pollard-Rho for prime factorization.

6.1.6 Prefix search correctness

The main loop scans integers $R \ge 191265$. It skips nonprimes and primes which are not split primes. For every split prime R, it computes

$$R(R) = \texttt{reachable\_events(R)}$$

and inserts these event identifiers into a global finite set

$$P_{\text{seen}}.$$

The counter `distinctPredictedEvents` is exactly

$$|P_{\text{seen}}|,$$

because each event identifier is counted only the first time its marker in `covered` changes from false to true.

The run stops at the first point where

$$|P_{\text{seen}}| > 95568.$$

The generated output is:

$$R = 191281, \qquad |P_{\text{seen}}| = 181052, \qquad |A| = 95568.$$

So the finite predicted event set is strictly larger than the actual side-labeled MP/PM event set.

6.1.7 Certificate implication

The mathematical routing theorem certified by the MP/PM event computation is: if a cofinal tail of midpoint-exceptional primes exists, then every event in the computed finite set

$$P_{\text{seen}}$$

is realized in the actual generated MP/PM event set A. So

$$|P_{\text{seen}}| \leq |A|.$$

But the computed values give

$$181052 = |P_{\text{seen}}| > 95568 = |A|,$$

a contradiction. So no such cofinal exceptional tail exists. This proves the Lean theorem

`GeneratedCertificate.external_no_cofinalExceptionTail`.

The only project-specific external declaration used to derive it is

`GeneratedCertificate.external_predictedEvents_realized_of_cofinalTail`.

6.2 Formal objects used in Section 5

`TwinPrimeExternal/Core.lean`

- `generatedMPPMCard`: the actual generated MP/PM event count 95568.
- `predictedEventCount`: the predicted event count 181052.
- `predictedEventCount_exceeds_generatedMPPMCard`: checks $95568 < 181052$.
- `CofinalExceptionTail`: the cofinal-tail hypothesis consumed by the route-realization bridge.
- `MidpointExceptionalPrime`: the exception predicate used in that cofinal tail.

`TwinPrimeExternal/RecursiveMPPMCertificate.lean`

- `left_row_residue5_identity`: the left-row residue-5 identity.
- `right_row_residue5_identity`: the right-row residue-5 identity.
- `quad_route_left_identity`: the left row algebra identity.

- `quad_route_right_identity`: the right row algebra identity.
- `encodeSideEvent`: encodes a coordinate and side label as one natural number.
- `encodeSideEvent_zero_ne_one`: proves the MP and PM side labels differ at a fixed coordinate.
- `encodeSideEvent_injective_of_side_lt_two`: proves injectivity for labels $0, 1$.
- `DescendedPrimeEdge`: the decreasing-edge predicate for descended primes.
- `DescendedPrimeEdge.decreases`: extracts the strict decrease.
- `DescendedPrimeEdge.irrefl`: rules out self-edges.
- `DescendedPrimeEdge.no_two_cycle`: rules out two-cycles.
- `RecursiveMPPMEventCertificate`: the Lean certificate shape.
- `RecursiveMPPMEventCertificate.predicted_card_exceeds_actual`: rewrites generated card fields into $95568 < 181052$.
- `RecursiveMPPMEventCertificate.false_of_cofinalTail`: the finite-set contradiction.
- `no_cofinalExceptionTail_of_recursiveMPPMEventCertificate`: packages the contradiction as a no-tail theorem.

`TwinPrimeExternal/GeneratedCertificate.lean`

- `firstOverflowPrime`: the first overflow split prime 191281.
- `firstOverflowSpan`: the generated prefix span.
- `firstOverflowSplitPrimes`: the number of split primes used at first overflow.
- `firstOverflowRoutedStarts`: the number of routed starts at first overflow.
- `E_a`: the actual event count 95568.
- `E_p`: the predicted event count 181052.
- `falsePredictedEventCount`: the generated count of false predicted events in the audit.
- `E_a_eq_cap`: identifies E_a with the core actual-count constant.
- `E_p_eq_core`: identifies E_p with the core predicted-count constant.
- `E_p_exceeds_E_a`: checks $E_a < E_p$.
- `core_predictedEventCount_exceeds_generatedMPPMCard`: transfers the generated inequality to the core constants.
- `actualEvents`: the finite actual event set represented by count.
- `actualEvents_card`: its generated cardinality proof.
- `predictedEvents`: the finite predicted event set represented by count.
- `predictedEvents_card`: its generated cardinality proof.
- `external_predictedEvents_realized_of_cofinalTail`: the external route-realization bridge.
- `recursiveMPPMEventCertificate`: instantiates the Lean certificate shape.
- `external_no_cofinalExceptionTail`: the derived no-tail theorem.

7 Main theorem

Theorem 7.1 (Arbitrarily large twin primes)**.**

$$\forall N,\ \exists p,\ N \le p \wedge \mathrm{Prime}(p) \wedge \mathrm{Prime}(p+2).$$

Proof. Assume, for contradiction, that twin primes are not arbitrarily large. Then there is some N such that no prime $p \ge N$ has $p+2$ prime. This implies BoundedTwinMids: take $M = N+1$. If

$$\mathrm{Prime}(m-1) \wedge \mathrm{Prime}(m+1)$$

held for some $m > M$, then $p = m-1$ would satisfy $p \ge N$ and $\mathrm{Prime}(p) \wedge \mathrm{Prime}(p+2)$, contradiction.

By the bounded-midpoint lemma, BoundedTwinMids gives a cofinal tail of MidpointExceptionalPrime primes. This contradicts the generated certificate theorem

`GeneratedCertificate.external_no_cofinalExceptionTail`.

So twin primes are arbitrarily large. □

The Lean proof of the last implication is theorem

`arbitrarily_large_twins_of_no_cofinalExceptionTail`.

The final theorem exported by the project is

`TwinPrimeExternal.arbitrarily_large_twins`.

7.1 Formal objects used in Section 6

`TwinPrimeExternal/Core.lean`

- `no_boundedTwinMids_of_no_cofinalExceptionTail`: turns the no-tail theorem into the negation of bounded twin midpoints.
- `arbitrarily_large_twins_of_no_cofinalExceptionTail`: proves the main endpoint from no cofinal exceptional tail.
- `arbitrarily_large_twins_of_not_boundedTwinMids`: the final elementary unboundedness step.
- `no_boundedTwinMids_of_no_certificateFirstException`: fallback endpoint from no first exception after the certificate gap.
- `arbitrarily_large_twins_of_no_certificateFirstException`: fallback infinitude theorem from that first-exception obstruction.
- `no_later_firstException_of_no_certificateFirstException`: pushes a no-first-exception certificate to later gaps.
- `no_observed_firstException_of_no_certificateFirstException`: specializes the previous theorem to the observed gap.

`TwinPrimeExternal/GeneratedCertificate.lean`

- `external_no_cofinalExceptionTail`: supplies the no-tail theorem used by the main endpoint.

`TwinPrimeExternal/Final.lean`

- `no_cofinalExceptionTail`: reexports the generated no-tail theorem.
- `not_boundedTwinMids`: derives unbounded midpoint twins.
- `arbitrarily_large_twins`: the final theorem.

References

[1] A. de Polignac, *Recherches nouvelles sur les nombres premiers*, Comptes rendus de l'Académie des sciences, 29:397–401, 1849.

[2] V. Brun, *La série dont les termes sont les inverses des nombres premiers jumeaux est convergente ou finie*, Bulletin des Sciences Mathématiques, 43:100–104, 124–128, 1919.

[3] J. R. Chen, *On the representation of a large even integer as the sum of a prime and the product of at most two primes*, Scientia Sinica, 16:157–176, 1973.

[4] Y. Zhang, *Bounded gaps between primes*, Annals of Mathematics, 179(3):1121–1174, 2014.

[5] J. Maynard, *Small gaps between primes*, Annals of Mathematics, 181(1):383–413, 2015.

[6] D. H. J. Polymath, *Variants of the Selberg sieve, and bounded intervals containing many primes*, Research in the Mathematical Sciences, 1:12, 2014.

[7] D. Shanks, *Five number-theoretic algorithms*, Congressus Numerantium, 7:51–70, 1973.

A The empirical gap certificate and midpoint-row gap checker

The repository also includes a finite gap checker:

`GeneratedGapCertificate.external_exceptionFree_to_certificateThreshold`.

It certifies that every prime

$$127 < r \leq 191264$$

has a midpoint-row twin witness. So any first exception after 127 must occur after 191264. The Lean theorem is

- `GeneratedGapCertificate.any_firstException_after_127_occurs_after_certificateThreshold`

This theorem is not a dependency of

- `TwinPrimeExternal.arbitrarily_large_twins`

It is included because it records the empirical starting point of the search: the generated finite contradiction certificate is applied beyond a verified no-exception window after the last observed exception 127.

A.1 Checker correctness

This appendix section proves the correctness of

`tools/check_cone_survivor_gap.cpp`.

A.2 Specification

The checker takes two integers $L < V$, defaulting to

$$L = 127, \qquad V = 191264.$$

It must verify:

$$\forall r,\ \mathrm{Prime}(r) \wedge L < r \leq V \Rightarrow \neg\,\mathrm{MidpointExceptionalPrime}(r).$$

Equivalently, for every prime $r \in (L, V]$, it must find an index

$$1 \leq h < r$$

such that both $rh - 1$ and $rh + 1$ are prime.

A.3 Prime enumeration

The function `primes_upto(V)` is the ordinary Eratosthenes sieve: it iterates $p = 2, \ldots, V$, emits p if not previously marked composite, and marks multiples $p^2, p^2 + p, \ldots$. The standard invariant is: after processing all primes $< p$, a number $n \geq p$ is marked if and only if it has a prime factor $< p$. So at the time p is inspected, it is unmarked if and only if it is prime. So the returned vector is exactly the list of primes up to V.

A.4 Reduction to indices divisible by six

Let $r > 3$ be prime. If h is odd, then rh is odd, so both

$$rh - 1, \qquad rh + 1$$

are even and larger than 2. So they cannot both be prime.

If $3 \nmid h$, then $rh \not\equiv 0 \pmod 3$, since $r \neq 3$. So one of $rh-1, rh+1$ is divisible by 3. In the checked range $r > 127$, this divisible neighbor is larger than 3, so it is composite. So any successful index h must be divisible by 6. The checker is complete when it searches

$$h = 6, 12, 18, \ldots < r.$$

A.5 The twin-witness predicate

The function `cone_survives(r,h,primes)` computes

$$n_- = rh - 1, \qquad n_+ = rh + 1,$$

and checks every prime $q < r$. It returns true exactly when no prime $q < r$ divides n_- or n_+.

Lemma A.1. *Let r be prime, $1 \leq h < r$, and $n \in \{rh - 1, rh + 1\}$. If no prime $q < r$ divides n, then n is prime.*

Proof. We have $n > 1$. Also

$$rh + 1 \leq r(r - 1) + 1 = r^2 - r + 1 < r^2,$$

so $n < r^2$. If n were composite, it would have a prime divisor

$$q \leq \sqrt{n} < r.$$

This contradicts the assumption that no prime $q < r$ divides n. So n is prime. □

So, if `cone_survives(r,h,primes)` returns true, the listed h witnesses MidpointTwinWitnessAt(r, h), and r is not a MidpointExceptionalPrime.

A.6 Output correctness

The checker loops over exactly the primes $r \in (L, V]$. For each such r, it searches every potentially successful index $h \equiv 0 \pmod 6$ with $h < r$. If it finds a twin witness, it records a row (r, h). If not, it records r in `failedPrimes`.

The generated run produced

$$\texttt{checkedPrimes} = 17236, \qquad \texttt{survivorRows} = 17236, \qquad \texttt{failedPrimes} = [].$$

So every prime in $(127, 191264]$ has a midpoint-row twin witness. This proves the external Lean declaration

`GeneratedGapCertificate.external_exceptionFree_to_certificateThreshold`.

A.7 Formal objects used in Appendix A

`TwinPrimeExternal/Core.lean`

- `ExceptionFreeUpTo`: the finite no-exception-window predicate.
- `FirstExceptionAfterLastObserved`: the first-exception record used by this auxiliary checker.
- `firstExceptionAfterLast_occurs_after_of_exceptionFreeUpTo`: the general theorem pushing a first exception past a checked window.
- `firstExceptionAfterLast_occurs_after_certificateThreshold_of_exceptionFreeUpTo`: the certificate-threshold specialization.

`TwinPrimeExternal/GeneratedGapCertificate.lean`

- `gapLast`: the generated last-exception bound.
- `gapCheckedTo`: the generated checked-to threshold.
- `gapSize`: the generated gap size.
- `checkedPrimes`: the number of checked primes in the gap run.
- `failedPrimes`: the generated failed-prime count.
- `gapLast_eq_lastObservedException`: checks the generated last bound.
- `gapCheckedTo_eq_certificateVerifiedTo`: checks the generated checked-to threshold.
- `failedPrimes_eq_zero`: checks that no failures were generated.
- `external_exceptionFree_to_certificateThreshold`: the auxiliary external gap declaration.
- `any_firstException_after_127_occurs_after_certificateThreshold`: derives the advertised first-exception lower bound.

B Lean audit

The audit command

```
#print axioms TwinPrimeExternal.arbitrarily_large_twins
```

reports:

- `propext`
- `Classical.choice`
- `Quot.sound`
- `TwinPrimeExternal.GeneratedCertificate.external_predictedEvents_realized_of_cofinalTail`

The certificate theorem

- `GeneratedCertificate.external_no_cofinalExceptionTail`

is derived in Lean from

- `no_cofinalExceptionTail_of_recursiveMPPMEventCertificate`

The auxiliary theorem

- `GeneratedGapCertificate.any_firstException_after_127_occurs_after_certificateThreshold`

depends on

- `GeneratedGapCertificate.external_exceptionFree_to_certificateThreshold`

It is not a dependency of the final theorem.

B.1 Formal objects used in Appendix B

`TwinPrimeExternal/Final.lean`

– `arbitrarily_large_twins`: the theorem inspected by `#printaxioms`.

`TwinPrimeExternal/GeneratedCertificate.lean`

– `external_predictedEvents_realized_of_cofinalTail`: the only project-specific declaration reported for the final theorem.

– `external_no_cofinalExceptionTail`: checked separately as a Lean-derived theorem from that declaration.

`TwinPrimeExternal/GeneratedGapCertificate.lean`

– `external_exceptionFree_to_certificateThreshold`: the auxiliary gap declaration that appears only when auditing the auxiliary gap theorem.

C Repository coverage map

This appendix covers every non-build file in this repository. The `.lake` directory, compiled C++ binaries, and LaTeX auxiliary files are build artifacts. The C++ source files in `tools/` carry the external certificate algorithms.

C.1 Lean files

- `TwinPrimeExternal.lean`: the aggregate import file for the Lean library. It imports the core definitions, recursive certificate model, generated certificates, and final theorem.
- `TwinPrimeExternal/Core.lean`: defines the numerical constants 127, 10^9, 191264, 191281, 95568, and 181052; defines observed gaps, first-exception records, cofinal exception tails, bounded twin midpoints, midpoint rows, midpoint witnesses, and midpoint-exceptional primes; proves the Lean chain from bounded twin midpoints to a cofinal exceptional tail and from no cofinal exceptional tail to arbitrarily large twin primes. Sections 2–4 and the main theorem section cover these definitions and lemmas. Section 6 lists the small fallback first-exception endpoints.
- `TwinPrimeExternal/RecursiveMPPMCertificate.lean`: formalizes the Lean-side recursive MP/PM certificate shape, the four row identities, and the finite-set cardinality contradiction. Section 1.2 and Section 5 cover this file.
- `TwinPrimeExternal/GeneratedCertificate.lean`: generated Lean file containing $E_a = 95568$, $E_p = 181052$, the finite event-set cardinality checks, the route-realization external declaration, and the derived no-tail theorem. Section 5 and the Lean audit cover this file.
- `TwinPrimeExternal/GeneratedGapCertificate.lean`: generated Lean file for the auxiliary no-exception window $(127, 191264]$. Appendix A covers this file. Its external declaration is not a dependency of the final theorem.
- `TwinPrimeExternal/Final.lean`: exports the final no-tail, unbounded-midpoint, and arbitrarily-large-twins endpoints. The main theorem section and Lean audit cover this file.

C.2 C++ tools and certificates

- `tools/search_recursive_prefix_threshold.cpp`: computes the recursive MP/PM closure and finds the first overflow $E_p > E_a$. Section 5 proves the intended algorithm.
- `tools/generate_external_certificate.cpp`: converts the search summary into `TwinPrimeExternal/GeneratedCertificate.lean` and the top-level `certificate.json` audit summary. Section 5 covers the generated theorem boundary.
- `tools/check_cone_survivor_gap.cpp`: finite midpoint-row gap checker for $(127, 191264]$. Appendix A covers its specification and correctness argument. The historical function name still contains `cone`; the paper terminology uses midpoint rows.

- `tools/print_routing_example.cpp`: prints the five-stage descent route displayed in Section 1.2.
- `certificates/generated_mppm_pressure_certificate.json`: finite generated Base/MP/PM input block consumed by the recursive MP/PM search. Section 5.1 covers its role.
- `certificates/generated_external_certificate_summary.json`: compact JSON summary of the recursive overflow computation. It records the values emitted into `GeneratedCertificate.lean`.
- `certificates/generated_gap_certificate_summary.json`: compact JSON summary of the finite midpoint-row gap checker. Appendix A covers its contents.
- `certificate.json`: top-level audit summary emitted by `generate_external_certificate.cpp`. It repeats the finite overflow values 191281, 17, 95568, 181052, and 115633.

C.3 Repository and paper support files

- `paper/twin_prime_external_certificate_endpoint.tex`: this paper.
- `paper/appendix_gap_certificate_stub.tex`: separate appendix stub for the auxiliary gap checker and its generated Lean certificate.
- `paper/twin_prime_external_certificate_endpoint.pdf`: rendered PDF output from the TeX source.
- `README.md`: command-oriented repository summary and regeneration notes.
- `lakefile.toml`: Lean package definition for `TwinPrimeExternal`.
- `lake-manifest.json`: Lake dependency manifest.
- `lean-toolchain`: Lean toolchain pin.
- `.gitignore`: excludes generated build clutter and local artifacts.